PORTFOLIO PROJECT #7 • MARCH 2026

MediaLens

AI-Powered Multi-Modal Content Intelligence Platform

Express.js | tRPC v11 | SvelteKit | TypeScript | Google Gemini AI | Drizzle ORM | PostgreSQL | MinIO | Meilisearch | Arctic OAuth 2.0 | Docker Compose | Playwright E2E | Tailwind CSS | Zod

AUTHOR

Malbin Jose

GITHUB

github.com/malbinjyc/media-lens

Project Report

March 5, 2026

Confidential • Portfolio Documentation

NAVIGATION

Table of Contents

MediaLens Project Report — Technical Documentation

1 Project Overview 3

2 Technology Stack 4

3 System Architecture 5

4 Key Features 6

5 Database Schema 7

6 API Reference (tRPC Routers) 8

7 Security Model 9

8 Project Structure 10

9 Skills & Portfolio Differentiation 11

10 Getting Started 12

62	20	4	9
SOURCE FILES	TRPC PROCEDURES	DB TABLES	NEW SKILLS

SECTION 01

Project Overview

What MediaLens is and why it was built

MediaLens is an AI-powered multi-modal content intelligence platform. Users upload images, PDFs, and text documents to a personal media library, organize them into color-coded projects, and leverage Google Gemini AI for deep analysis, auto-summarization, auto-tagging, and natural-language Q&A. Meilisearch provides instant full-text search across all media metadata.

Problem Statement

Professionals and researchers routinely accumulate large collections of mixed media — scanned documents, screenshots, PDFs, images — that are difficult to organize, search, and extract insight from. Existing tools either silo file types or require manual tagging. MediaLens solves this by combining AI analysis with a structured media library and instant search into a single, cohesive platform.

Solution

📸

Unified Media Library

Accept images (JPEG, PNG, WebP), PDFs, and plain-text documents through a single drag-and-drop interface with presigned MinIO uploads for security and scalability.

🤖

AI-Powered Analysis

Google Gemini provides image description, OCR, document summarization, auto-tagging, and open-ended Q&A through a unified multi-modal API.

📁

Project Organization

Group related media into color-coded projects with descriptions. Each project acts as a workspace providing context for AI interactions.

🔍

Instant Full-Text Search

Meilisearch indexes all media titles, descriptions, AI analysis text, and tags, enabling millisecond full-text and faceted search across the entire library.

Target Use Cases

📋 **Research & Document Management**
Upload scanned articles, extract key insights via AI, and search across your entire research library instantly.

🎨 **Design Asset Organization**
Tag and organize visual assets by project, with AI auto-describing image content for better discoverability.

💼 **Business Intelligence**

Upload reports and images; ask natural language questions about their content without manual reading.

SECTION 02

Technology Stack

Every library, framework, and service powering the platform

🖥️ SERVER

Express.js · TypeScript · tRPC v11 · Drizzle ORM · PostgreSQL · Arctic (OAuth 2.0) · Google Gemini AI · MinIO (S3) · Meilisearch · Zod · Helmet · CORS

🌐 CLIENT

SvelteKit · TypeScript · Tailwind CSS · Lucide Svelte · tRPC Client · Vite

🛠️ INFRASTRUCTURE

Docker Compose · PostgreSQL 16 · Meilisearch · MinIO

🧪 TESTING

Playwright E2E

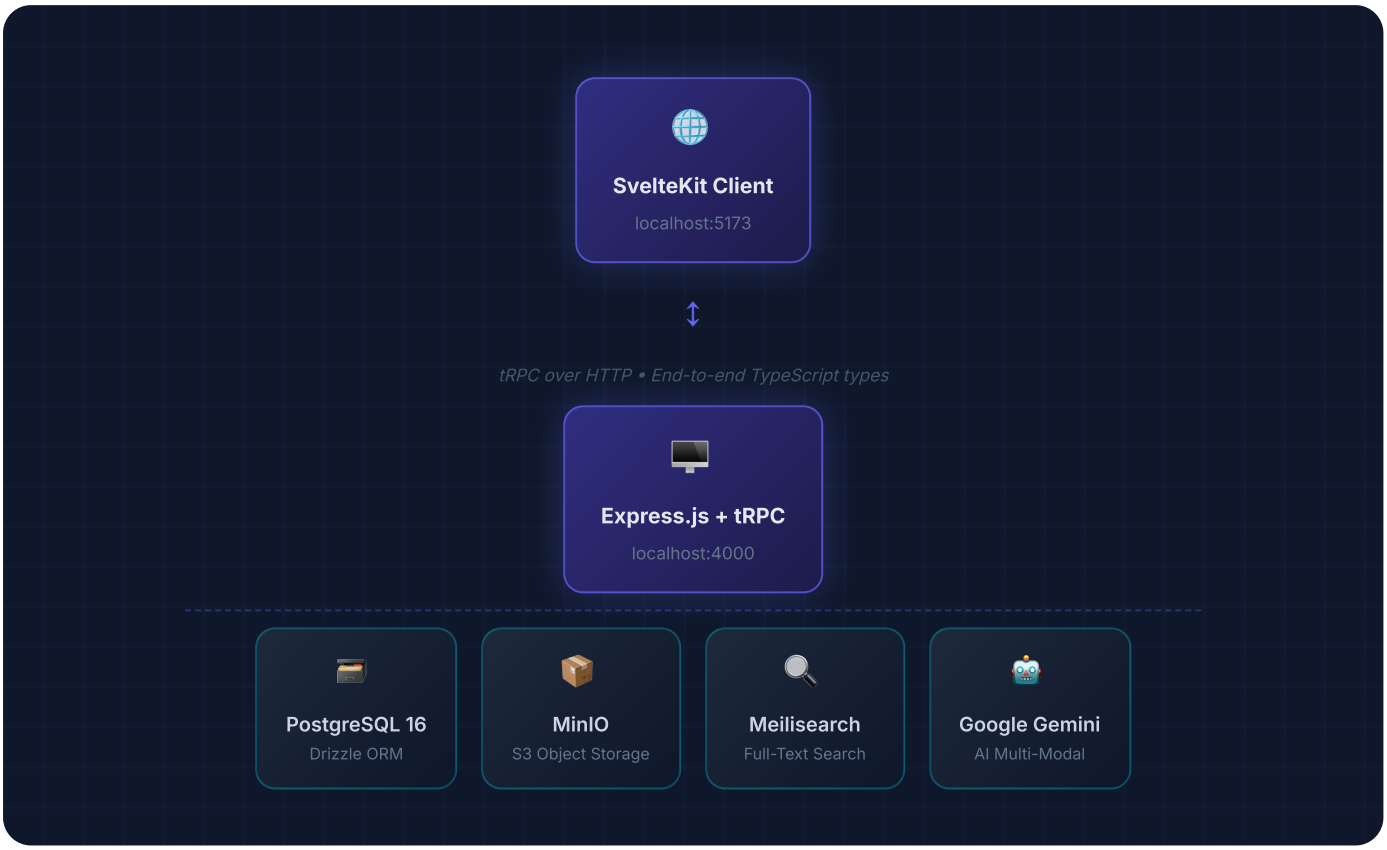
Technology Rationale

TECHNOLOGY	ROLE	WHY CHOSEN
Express.js	HTTP Server	Lightweight, universal; works as tRPC adapter host
tRPC v11	API Layer	End-to-end TypeScript type safety without schema generation
SvelteKit	Frontend Framework	Compiler-based reactivity, minimal runtime, excellent DX
Drizzle ORM	Database ORM	TypeScript-first, SQL-like queries, lightweight and fast
Arctic	OAuth 2.0	Lightweight library supporting 60+ OAuth providers
Google Gemini	AI / ML	Unified multi-modal API: vision, text, OCR in one model
MinIO	Object Storage	Self-hosted S3-compatible storage; presigned URL support
Meilisearch	Search Engine	Instant typo-tolerant full-text search with faceting
Zod	Validation	Runtime schema validation that integrates natively with tRPC
Playwright	E2E Testing	Cross-browser automation with first-class TypeScript support

SECTION 03

System Architecture

How all components connect and interact



Data Flow

STEP	ACTION	COMPONENTS INVOLVED
1	User authenticates via Google or GitHub OAuth	SvelteKit → Express → Arctic → PostgreSQL
2	Client requests a presigned upload URL	tRPC `media.getUploadUrl` → MinIO SDK
3	Browser uploads file directly to MinIO	Browser → MinIO (presigned PUT)
4	Client confirms upload; server registers media record	tRPC `media.create` → PostgreSQL + Meilisearch index
5	AI analysis triggered on demand	tRPC `ai.analyze` → MinIO (fetch) → Gemini API
6	Analysis result stored; search index updated	PostgreSQL (aiAnalysis) + Meilisearch (reindex)
7	User searches media	tRPC `search.query` → Meilisearch → results

All infrastructure services (PostgreSQL, MinIO, Meilisearch) run as Docker containers defined in a single `docker-compose.yml` at the project root, enabling one-command local setup.

SECTION 04

Key Features

Detailed walkthrough of all platform capabilities

🔑 **OAuth 2.0 Authentication**

Passwordless sign-in via Google and GitHub using the Arctic OAuth library. Sessions are stored as server-side tokens in PostgreSQL with configurable expiry. No passwords are stored.

📷 **Media Library with Drag-and-Drop Upload**

Users upload images (JPEG, PNG, WebP), PDFs, and plain-text files. Upload goes directly from the browser to MinIO using server-generated presigned URLs, keeping large binary data off the API server. File type and size are validated server-side.

📁 **Project Organization**

Media is grouped into named, color-coded projects with optional descriptions. Projects serve as contextual workspaces. Deleting a project disassociates (not deletes) the media within it.

👁️ **AI Image Analysis & OCR**

The `ai.analyze` procedure fetches the file from MinIO and sends it to Google Gemini with a structured prompt. For images, Gemini describes visual content and extracts embedded text. Results are stored in the `aiAnalysis` field and indexed for search.

📄 **Document Summarization**

PDF and text documents are summarized by Gemini via the `ai.summarize` procedure. Summaries capture key points, topics, and entities, making long documents searchable by their meaning rather than just keywords.

🏷️ **Auto-Tagging**

The `ai.tag` procedure asks Gemini to generate a curated list of relevant tags for any uploaded media item. Tags are stored as a PostgreSQL text array and indexed in Meilisearch for faceted filtering.

💬 **Multi-Modal Q&A**

Users pose free-form questions about any media item via `ai.ask`. Gemini receives both the file and the question, returning a contextual answer. This works across images, PDFs, and documents.

🔍 **Full-Text + Faceted Search**

Meilisearch indexes every media record's title, description, AI analysis, and tags. Search is typo-tolerant, instant (<50ms), and supports faceting by project, MIME type, and tag. The `search.query` tRPC procedure exposes this.

🔗 **End-to-End Type Safety**

tRPC infers TypeScript types from server-side Zod schemas and propagates them to the SvelteKit client automatically. No REST contracts, no code generation — types are always in sync at compile time.

SECTION 05

Database Schema

PostgreSQL tables defined with Drizzle ORM

The database consists of four tables. Drizzle ORM manages schema definitions, migrations, and type inference. All primary keys use UUIDs generated by PostgreSQL's `gen_random_uuid()`. Timestamps include timezone information.

users

Identity

id	email	name	avatarUrl	provider	providerId	createdAt
uuid PK	text UNIQUE	text	text	enum(google|github)	text	timestamptz

sessions

Auth

id	userId	token	expiresAt	createdAt
uuid PK	uuid FK → users	text UNIQUE	timestamptz	timestamptz

projects

Organization

id	userId	name	description	color	createdAt	updatedAt
uuid PK	uuid FK → users	text	text	text (hex)	timestamptz	timestamptz

media

Core Entity

id	userId	projectId	filename	mimeType	size	storageKey	title
uuid PK	uuid FK → users	uuid FK → projects	text	text	bigint	text	text

description	aiAnalysis	tags	status	createdAt	updatedAt
text	text	text[]	enum(pending|analyzed|failed)	timestamptz	timestamptz

Foreign keys use **CASCADE DELETE** on user deletion, ensuring all sessions, projects, and media are automatically removed when a user account is deleted. The `media.projectId` uses **SET NULL** on project deletion, preserving media files when a project is removed.

SECTION 06

API Reference

Complete tRPC router and procedure documentation — 5 routers, 20 procedures

ROUTER	PROCEDURE	TYPE	DESCRIPTION	AUTH
auth	getLoginUrl	mutation	Generate OAuth authorization URL for Google or GitHub	Public
auth	callback	mutation	Exchange OAuth code for session token; upsert user	Public
auth	me	query	Return authenticated user profile	Required
auth	logout	mutation	Invalidate session token in database	Required
media	list	query	Paginated list of user's media; filter by project	Required
media	get	query	Fetch single media record with presigned download URL	Required
media	getUploadUrl	mutation	Generate presigned MinIO PUT URL for direct browser upload	Required
media	create	mutation	Register uploaded file; index in Meilisearch	Required
media	update	mutation	Update media title, description, project, tags	Required
media	delete	mutation	Delete database record and MinIO object	Required
projects	list	query	Return all projects belonging to the current user	Required
projects	get	query	Fetch project details with associated media list	Required
projects	create	mutation	Create a new project with name, description, color	Required
projects	update	mutation	Rename or recolor an existing project	Required
projects	delete	mutation	Delete project; sets media.projectId to NULL	Required
ai	analyze	mutation	Gemini multi-modal analysis (vision + OCR for images)	Required
ai	summarize	mutation	Generate document summary via Gemini	Required
ai	tag	mutation	Auto-generate tag list from media content	Required
ai	ask	mutation	Natural language Q&A about a media item	Required
search	query	query	Full-text + faceted search via Meilisearch index	Required

SECTION 07

Security Model

Defence-in-depth approach across all layers

OAuth 2.0 Authentication

No passwords stored. Google and GitHub OAuth via Arctic library. Authorization codes are single-use and short-lived. Redirect URIs are validated server-side.

Server-Side Session Tokens

Session tokens are stored in PostgreSQL with expiry timestamps. Tokens are validated on every authenticated request via tRPC middleware. Logout invalidates the token immediately.

tRPC Middleware Auth Guard

A `protectedProcedure` middleware enforces authentication on all 18 non-public procedures. Unauthenticated calls return a typed `UNAUTHORIZED` error.

Zod Input Validation

Every tRPC procedure input is validated at runtime with Zod schemas before reaching business logic. Invalid inputs are rejected with typed error responses.

Resource Ownership Checks

All read and write operations verify that the resource belongs to the authenticated user before returning data or applying mutations. Prevents cross-user data access.

Helmet + CORS Headers

Express Helmet sets secure HTTP headers (CSP, HSTS, X-Frame-Options, etc.). CORS is configured to allow only the SvelteKit client origin.

Presigned MinIO URLs

Files are uploaded directly from the browser to MinIO using time-limited presigned PUT URLs. The API server never touches file bytes, eliminating upload proxy attack surface.

File Type & Size Validation

Server validates MIME type and file size before issuing a presigned URL. Only allowed types (image/jpeg, image/png, image/webp, application/pdf, text/plain) are accepted.

Docker Non-Root User

Both server and client Dockerfiles run as non-root users, reducing the blast radius of any container escape vulnerability.

Secrets via Environment Variables

All secrets (OAuth client IDs/secrets, Gemini API key, database URL, MinIO credentials) are injected at runtime via environment variables. Never hardcoded or committed.

SECTION 08

Project Structure

62 source files organized across three top-level directories

server/

Express.js + tRPC API. TypeScript source in `src/`. Drizzle config and migrations. Dockerfile for production.

client/

SvelteKit application. Tailwind CSS. Vite build tooling. Dockerfile for production deployment.

e2e/

Playwright end-to-end test suite. Playwright config. Test files covering critical user flows.

```
media-lens/
├── docker-compose.yml          # PostgreSQL 16, MinIO, Meilisearch
├── server/
│   ├── Dockerfile
│   ├── drizzle.config.ts
│   ├── package.json
│   ├── tsconfig.json
│   └── src/
│       ├── index.ts              # Express app entry point
│       ├── router.ts             # Root tRPC router
│       ├── trpc.ts               # tRPC init + middleware
│       ├── db/
│       │   ├── schema.ts          # Drizzle table definitions
│       │   ├── index.ts           # DB connection pool
│       │   └── migrate.ts        # Migration runner
│       ├── routers/
│       │   ├── auth.router.ts
│       │   ├── media.router.ts
│       │   ├── projects.router.ts
│       │   ├── ai.router.ts
│       │   └── search.router.ts
│       └── services/
│           ├── ai.service.ts       # Gemini API wrapper
│           ├── search.service.ts   # Meilisearch wrapper
│           └── storage.service.ts  # MinIO presigned URL logic
├── client/
│   ├── Dockerfile
│   ├── svelte.config.js
│   ├── tailwind.config.ts
│   ├── vite.config.ts
│   └── src/                    # SvelteKit routes + components
└── e2e/
    ├── playwright.config.ts
    └── tests/                 # E2E test files
```

SECTION 09

Skills & Portfolio Differentiation

How MediaLens fills remaining tech stack gaps in the portfolio

Portfolio Project #7

MediaLens was specifically designed as the seventh portfolio project to demonstrate proficiency in technologies not yet covered by previous projects. Each technology choice represents a deliberate skill acquisition goal.

Express.js
Node.js HTTP server & middleware composition

tRPC
Type-safe API without REST or GraphQL overhead

SvelteKit
Compiler-first reactive frontend framework

Google Gemini
Multi-modal AI for vision, OCR, and NLP

Drizzle ORM
TypeScript-native SQL ORM with zero runtime overhead

Arctic OAuth
Lightweight OAuth 2.0 provider integrations

MinIO
Self-hosted S3-compatible object storage

Meilisearch
Instant typo-tolerant full-text search engine

Playwright
Cross-browser end-to-end test automation

Architecture Patterns Demonstrated

Presigned URL Upload Pattern
Offloads binary upload from API server to object storage directly, a critical pattern for scalable file-handling systems used in production at scale.

Multi-Modal AI Integration
Demonstrates fetching binary content from storage, encoding for AI API consumption, and persisting structured results — a real production AI pipeline.

Search Index Synchronization
Maintaining a secondary search index (Meilisearch) in sync with the primary database (PostgreSQL) on every create, update, and delete operation.

End-to-End Type Safety
tRPC eliminates API contract drift by sharing types directly from server to client. Zod schemas serve as both runtime validators and TypeScript type sources.

SECTION 10

Getting Started

Local development setup and environment configuration

Prerequisites

- Node.js 20+
- Docker & Docker Compose
- Google Cloud OAuth credentials
- GitHub OAuth App
- Google Gemini API key

Step 1 — Clone & Configure

```
# Clone the repository git clone https://github.com/malbinjvc/media-lens.git cd media-lens # Copy environment template cp .env.example .env # Edit .env with your credentials: # DATABASE_URL, GOOGLE_CLIENT_ID/SECRET, # GITHUB_CLIENT_ID/SECRET, GEMINI_API_KEY, # MINIO_ROOT_USER, MINIO_ROOT_PASSWORD
```

Step 2 — Start Infrastructure

```
# Starts PostgreSQL 16, Meilisearch, MinIO docker compose up -d postgres meilisearch minio
```

Step 3 — Run the Server

```
cd server npm install npm run db:generate # Generate Drizzle migration files npm run db:migrate # Apply migrations to PostgreSQL npm run dev # Start on http://localhost:4000
```

Step 4 — Run the Client

```
cd ../client npm install npm run dev # Start on http://localhost:5173
```

Step 5 — Run E2E Tests

```
cd ../e2e npx playwright test
```

Production Deployment

```
# Build and run all services via Docker Compose docker compose up -d --build
```

The MinIO console is available at **http://localhost:9001**. Create a bucket named `medialens` after first startup, or configure the `MINIO_BUCKET` environment variable to match your bucket name.

Repository

https://github.com/malbinjvc/media-lens

Author: Malbin Jose • March 2026